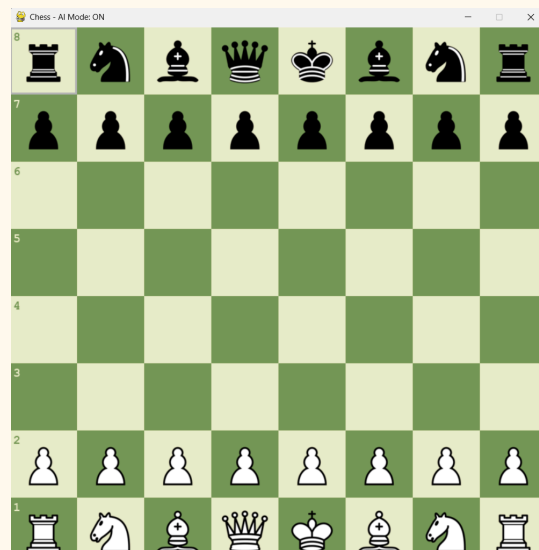


Build a Playable Chess Game with a Simple AI (Python + Pygame + OOP)

By Jessie Yeung



INTRODUCTION

This guide walks you through building a fully functional chess game with a basic AI opponent using Python, Pygame, and Object-Oriented Programming. By the end of this project, you will have a locally runnable chess application featuring a graphic chess board, draggable pieces, rule-based movement, and a minimax-algorithm AI opponent.

You will structure the program into distinct components, Board, Game/Engine, Pieces, Dragger, and AI, to demonstrate modular design. Each section builds on the previous one, so complete steps sequentially and test frequently.

Tools and Materials

- Python 3.10 or later
- Pygame library
- Visual Studio Code (recommended) or another IDE
- Terminal or command prompt

Warnings, Cautions, and Safety Notes

- Avoid mixing coordinate systems.
 - Decide whether your board uses (row, column) or (x, y) and remain consistent throughout the project.
- Do not reload images every frame.
 - Load piece images once at startup to prevent performance issues.
- Test after every major section.
 - Do not move forward if the previous section is not functioning properly.
- Be cautious when implementing minimax.
 - Deep recursion can heavily slow performance. Start with depth = 2.
- Save your work often and use version control if possible to avoid losing major progress.

Quick Start Guide

If you want to see immediate progress before building everything:

1. Install Pygame.
2. Create a Pygame window.
3. Draw an 8×8 grid.
4. Place simple colored circles instead of full piece logic.
5. Confirm that the window runs without errors.

Once this works, begin implementing full functionality following the detailed steps below.

Step-by-Step Instructions

Step 1: Set Up the Environment

1. Create a new project folder named pygame-chess.
2. Create subfolders named src and assets.
3. Create a virtual environment (optional but recommended).
4. Install Pygame.
5. Place chess piece PNG files inside the assets folder.

Run your environment to confirm Pygame installs successfully.

```
Collecting pygame==2.6.1 (from -r requirements.txt (line 1))
  Using cached pygame-2.6.1-cp313-cp313-win_amd64.whl.metadata (13 kB)
Using cached pygame-2.6.1-cp313-cp313-win_amd64.whl (10.6 MB)
Installing collected packages: pygame
Successfully installed pygame-2.6.1
```

Step 2: Create the Main Window and Game Loop

1. Import Pygame and initialize it.
2. Define window dimensions (e.g., 800×800).
3. Define square size as window size divided by 8.
4. Create the Pygame window.
5. Implement the main game loop:
 - Handle events
 - Update game state
 - Draw everything
 - Control frame rate

Run the program and confirm a blank window appears.

```

72         while True:
73             # Always reference current game state
74             game = self.game
75             board = self.game.board
76             dragger = self.game.dragger
77
78             if self.state == 'menu':
79                 self.show_menu(screen)
80                 for event in pygame.event.get():
81                     if event.type == pygame.MOUSEBUTTONDOWN:
82                         pos = event.pos
83                         if self.pvp_button.collidepoint(pos):
84                             game.ai_mode = False
85                             self.state = 'game'
86                         elif self.ai_button.collidepoint(pos):
87                             game.ai_mode = True
88                             self.state = 'game'
89                     elif event.type == pygame.KEYDOWN:
90                         if event.key == pygame.K_p:
91                             game.ai_mode = False
92                             self.state = 'game'
93                         elif event.key == pygame.K_a:
94                             game.ai_mode = True
95                             self.state = 'game'
96                     elif event.type == pygame.QUIT:
97                         pygame.quit()
98                         sys.exit()

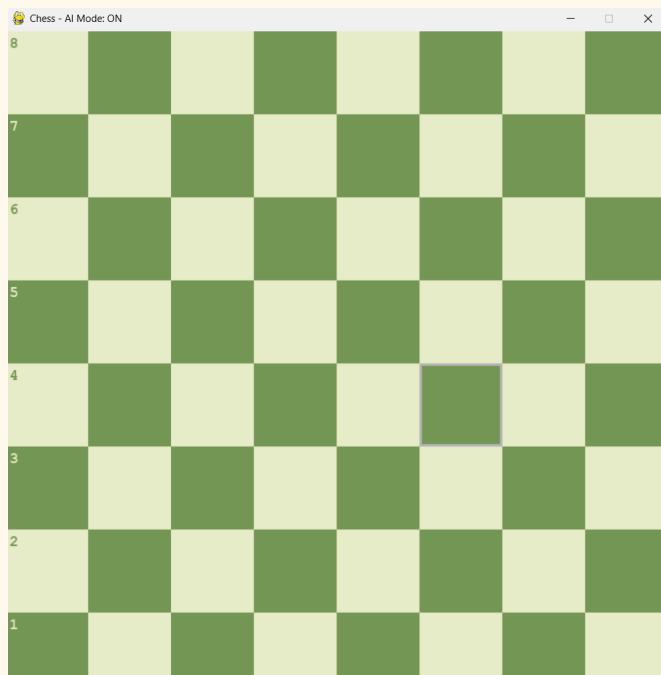
```

Sample code for the infinite game loop

Step 3: Implement the Board Class

1. Create a Board class inside src/board.py.
2. Initialize an 8×8 data structure to represent squares.
3. Store piece references inside board positions.
4. Create a method to set up starting positions.
5. Create a method to move pieces between squares.

Test by printing the board to the console.



The empty board should have identical squares and aligned, similar to this board (you may not have color yet).

Step 4: Create Piece Classes Using Inheritance

1. Create a base Piece class with:
 - color
 - name
 - value
2. Create subclasses:
 - Pawn
 - Rook
 - Knight
 - Bishop
 - Queen
 - King
3. Implement movement logic for each piece.
4. Ensure move generation:
 - Rejects off-board moves
 - Stops sliding pieces when blocked

- Prevents capturing same-color pieces

Test move generation using printed coordinates before integrating graphics.

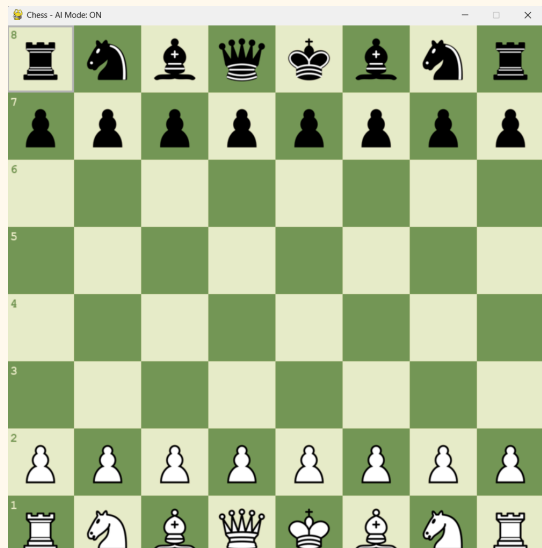
```
class Pawn(Piece):  
    def __init__(self, color):  
        self.dir = -1 if color == 'white' else 1  
        self.en_passant = False  
        super().__init__('pawn', color, 1.0)  
  
class Knight(Piece):  
    def __init__(self, color):  
        super().__init__('knight', color, 3.0)  
  
class Bishop(Piece):  
    def __init__(self, color):  
        super().__init__('bishop', color, 3.001)
```

Here is an example of the Pawn, Knight, and Bishop piece.

Step 5: Render the Board and Pieces

1. Draw alternating colored squares.
2. Load piece images once at startup.
3. Scale images to match square size.
4. Draw each piece based on board position.

Run the program and confirm a full chessboard displays correctly.



Here is a finished board with all the pieces.

Step 6: Add Mouse Interaction with a Dragger

1. Create a Dragger class.
2. Detect mouse button down events.
3. Identify which square was clicked.
4. Allow dragging only if the piece matches the current turn.
5. Update piece position visually while dragging.
6. On mouse release:
 - Convert mouse coordinates to board square
 - Validate the move
 - Update the board
 - Switch turns

Test by moving pieces manually.

```

def update_blit(self, surface):
    # texture
    self.piece.set_texture(size=128)
    texture = self.piece.texture
    # img
    try:
        img = pygame.image.load(texture)
        # rect
        img_center = (self.mouseX, self.mouseY)
        self.piece.texture_rect = img.get_rect(center=img_center)
        # blit
        surface.blit(img, self.piece.texture_rect)
    except pygame.error:
        print(f"Warning: Could not load dragged piece image {texture}")
        # Draw a placeholder circle for dragged piece
        color = (255, 0, 0) if self.piece.color == 'red' else (0, 0, 255)
        pygame.draw.circle(surface, color, (self.mouseX, self.mouseY), 30)

```

Step 7: Manage Game State and Turn Logic

1. Create a Game or Engine class.
2. Store:
 - current turn
 - selected piece
 - legal moves
3. Prevent illegal moves.
4. Switch turns only after valid moves.
5. Detect basic end condition (optional: king captured).

Test that white and black alternate turns properly.


```
def next_turn(self):
    self.next_player = 'white' if self.next_player == 'black' else 'black'

    # Check for game end conditions
    opponent = 'white' if self.next_player == 'black' else 'black'
    if self.board.is_checkmate(self.next_player):
        self.game_over = True
        self.winner = opponent # The player who just moved won
        print(f"Checkmate! {opponent.capitalize()} wins!")
    elif self.board.is_stalemate(self.next_player):
        self.game_over = True
        self.winner = 'draw'
        print("Stalemate! It's a draw!")
```

Sample code for next turn.

Step 8: Implement the AI (Minimax Algorithm)

1. Create an AI class.
2. Implement a board evaluation function:
 - Assign numeric values to pieces.
3. Generate all possible moves for a given side.
4. Implement minimax:
 - Base case: depth = 0 → evaluate board
 - Recursive case:
 - Maximize for AI
 - Minimize for opponent
5. Apply and undo moves during recursion.
6. Call AI after the human player moves.
7. Limit depth to 2 or 3.

Test that the AI selects legal moves and responds automatically.

```

if maximizing_player:
    max_eval = -float('inf')
    best_move = None
    best_moves = [] # Track all moves with the best score
    moves = get_all_moves(board, color)
    # Sort moves: captures first (simple heuristic)
    moves.sort(key=lambda m: 1 if board.squares[m.final.row][m.final.col].has_piece() else 0, reverse=True)
    for move in moves:
        # Make a copy of the board
        temp_board = copy.deepcopy(board)
        temp_piece = temp_board.squares[move.initial.row][move.initial.col].piece
        temp_board.move(temp_piece, move, testing=True)
        eval_score, _ = minimax(temp_board, depth - 1, alpha, beta, False, color)
        if eval_score > max_eval:
            max_eval = eval_score
            best_moves = [move] # New best score, reset list
        elif eval_score == max_eval:
            best_moves.append(move) # Equal score, add to list
        alpha = max(alpha, eval_score)
        if beta <= alpha:
            break
    # Randomly choose among equally good moves to avoid repetition
    best_move = random.choice(best_moves) if best_moves else None
    return max_eval, best_move

```

Sample code for maximizing AI.

Glossary

Board State – The current arrangement of all pieces.

Game Loop – The repeating cycle that updates and redraws the game.

Separation of Concerns – Dividing a program into distinct sections that each handle one responsibility.

Minimax – Algorithm that chooses moves assuming both players play optimally.

Evaluation Function – Method that assigns a numerical score to a board position.

Recursion – Function calling itself with modified input.

Frame Rate (FPS) – Number of times the screen updates per second.

Conclusion

You have now built a functional chess game with a graphical interface and a minimax AI opponent. Throughout this project, you implemented object-oriented design, state transitions, and a recursive decision-making algorithm.

If the AI feels weak, increase the search depth or add alpha–beta pruning. If performance drops, optimize move generation or limit recursion. As next steps, consider implementing check/checkmate detection, a move history log, or a difficulty selector.